

Musterlösung — Übungsklausur 1

Deep Learning 1 · TU Berlin

1 Multiple Choice

1. Was ist eine Gate-Funktion in einem LSTM?

✓ (b) Eine Multiplikation mit einer Sigmoid-Aktivierung

- (a) Softmax über Zeitschritte
- (c) ReLU der Hidden States
- (d) Lineare Projektion

2. Wie viele Parameter hat `torch.nn.Conv2d(3, 20, 5)`?

✓ (b) 1520 ($= 20 \times 3 \times 5 \times 5 + 20 = 1500 + 20$)

- (a) 1500 — vergisst Bias
- (c) 300
- (d) 6020

3. Was beschreibt die Konditionszahl der Hessematrix?

✓ (b) Verhältnis größter zu kleinster Eigenwert — Maß für Konvergenzgeschwindigkeit

- (a) Lernrate
- (c) Trainingsiterationen
- (d) Train-Test-Gap

4. Perceptron-Algorithmus $\equiv$ Gradientenabstieg auf welchem Loss?

✓ (c) Perceptron Loss: $\max(0, -y \cdot t)$

- (a) Squared Loss
- (b) Log-Loss
- (d) Hinge Loss

5. Welche Operation führt zu translationaler Invarianz in CNNs?

✓ (c) Pooling (z.B. Max Pooling)

- (a) Convolution $\rightarrow$ nur Equivariance
- (b) Batch Normalization
- (d) Residual Connection

6. Vorteil zentrierter Daten?

✓ (b) Konditionszahl der Hessematrix $\rightarrow 1$, garantiert schnellere Konvergenz

- (a) Gradienten null
- (c) Weniger Parameter
- (d) Dropout überflüssig

7. Was ist Double Descent?

✓ (b) Generalisierungsfehler sinkt erneut bei sehr großer Modellkomplexität

- (a) Zwei lokale Minima
- (c) Zwei Lernraten
- (d) Regularisierungsstrategie

8. Korrekte Aussage zu Dropout?

✓ (a) Dropout multipliziert jeden Eingang mit Bernoulli(p)-Zufallsvariable

- (b) Erhöht deterministisch Gewichte
- (c) Nur beim Testing
- (d) Ersetzt Aktivierungsfunktion

2 Theorie — RNN Backpropagation

(a) Berechnungsgraph

Knoten: $x_t, x_{t-1}, h_t=0, w$. Kanten: $h_t = \tanh(x_t \cdot w + h_{t-1})$, $y = h_t$, $E = (y-t)^2$.

Graph: $x_t \rightarrow [x_t \cdot w] \rightarrow [x_t \cdot w + h_{t-1}] \rightarrow [\tanh] \rightarrow h_t \rightarrow [h_t = y] \rightarrow [(y-t)^2] \rightarrow E$

(b) $\partial E / \partial y$

$$\partial E / \partial y = 2(y - t)$$

(c) Kettenregel für $\partial E / \partial w$

w taucht in h_t (direkt über $x_t \cdot w$) und in h_{t-1} (direkt über $x_{t-1} \cdot w$) auf:

$$\partial E / \partial w = (\partial E / \partial h_t) \cdot (\partial h_t / \partial w) + (\partial E / \partial h_{t-1}) \cdot (\partial h_{t-1} / \partial h_t) \cdot (\partial h_t / \partial w)$$

$$\partial h_t / \partial w = (1 - \tanh^2(x_t \cdot w + h_{t-1})) \cdot x_t$$

$$\partial h_{t-1} / \partial h_t = (1 - \tanh^2(x_{t-1} \cdot w + h_{t-2}))$$

$$\partial h_{t-1} / \partial w = (1 - \tanh^2(x_{t-1} \cdot w + h_{t-2})) \cdot x_{t-1}$$

(d) $\partial E / \partial w$ explizit

$$\partial E / \partial w = 2(y-t) \cdot [(1-h_t^2) \cdot x_t + (1-h_{t-1}^2) \cdot (1-h_t^2) \cdot x_{t-1}]$$

→ **Zwei Terme durch BPTT: direkter Term (aktueller Zeitschritt) + propagierter Term (zurück zu h_{t-1})**

3 Theorie — Verlustfunktionen

(a) Squared vs. Absolute Loss

Squared Loss $\mathcal{L} = (y-t)^2$: Vorteile: differenzierbar überall, tolerant gegenüber kleinen Fehlern. Nachteile: sehr sensitiv gegenüber Ausreißern (quadratische Bestrafung).

Absolute Loss $\mathcal{L} = |y-t|$: Vorteile: robuster gegenüber Ausreißern. Nachteile: nicht differenzierbar bei $y=t$ (Subgradient), gleiche Sensitivität für kleine und große Fehler.

(b) Log-Cosh Loss

$$\mathcal{L}(y, t) = (1/\beta) \cdot \log(\cosh(\beta \cdot (y-t)))$$

Verhält sich wie MSE für kleine Fehler ($\approx$ quadratisch) und wie MAE für große Fehler ($\approx$ linear). Überall differenzierbar, nur mild von Ausreißern beeinflusst. Entspricht der Hyperbolic-Secant-Verteilung.

(c) Maximum Likelihood $\leftrightarrow$ Squared Loss

Annahme: $t \sim N(\mu=f(x), \sigma^2)$. Log-Likelihood: $\log p(t|\mu, \sigma) = -(t-\mu)^2/(2\sigma^2) - \log(\sqrt{2\pi}\sigma)$.

Bei konstantem σ : Maximierung der Log-Likelihood $\equiv$ Minimierung von $(t-\mu)^2 =$ Squared Loss.

→ **Gaussian → Squared Loss · Laplace → Absolute Loss · Hyp. Secant → Log-Cosh**

4 Theorie — Regularisierung

(a) Weight Decay

Fügt dem Loss einen Term $\lambda \cdot \theta^2$ hinzu. Gradient: $\partial E_{\text{reg}} / \partial \theta = \partial E / \partial \theta + 2\lambda \cdot \theta$. Kleine Gewichte werden bevorzugt, sofern sie nicht für die Vorhersage notwendig sind. Hohes $\lambda \rightarrow$ starke Regularisierung (mehr Kontraktion).

(b) Early Stopping

Algorithmus: (1) Trainiere das Netz iterativ. (2) Beobachte den Validierungsfehler nach jeder Epoche. (3) Speichere Snapshot bei minimalem Validierungsfehler. (4) Stoppe Training bei Anstieg des Validierungsfehlers.

Regularisierung: Frühere Modelle haben effektiv weniger Komplexität (kürzere Optimierungstrajektorie $\approx$ kleinere Gewichte).

(c) Spurious Correlations

Definition: Task-irrelevante Input-Features sind zufällig mit relevanten Features in P korreliert — Modell lernt, Entscheidungen darauf zu basieren.

Beispiel: Pferde-Klassifikator lernt Copyright-Wasserzeichen als Merkmal, weil alle Pferdebilder aus derselben Quelle kamen.

Mitigierung 1: Modell inspizieren (XAI), Einheiten die auf das Artefakt reagieren identifizieren und entfernen.

Mitigierung 2: Artefakt manuell aus den Trainingsdaten entfernen.

5 Programmierung — CNN in PyTorch

(a) CNN-Klasse

```
import torch.nn as nn

class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2))
        self.classifier = nn.Linear(32*8*8, 10) # für 32×32 Input
    def forward(self, x):
        x = self.features(x)
        x = x.flatten(1)
        return self.classifier(x)
```

(b) Trainings-Loop

```
for epoch in range(num_epochs):
    for x, t in dataloader:
        optimizer.zero_grad() # 1. Gradienten nullsetzen
        y = model(x) # 2. Vorwärtsthroughlauf
        loss = criterion(y, t) # 3. Loss berechnen
        loss.backward() # 4. Rückwärtsthroughlauf
        optimizer.step() # 5. Parameter aktualisieren
```